

Group 12 - Image2GPS Final Project*Team Members: Ryan Zhou**Project: IMG2GPS*

1 Submission Links

- Colab Notebook: [Open Notebook](#)
- model.py: [Open model.py](#)
- preprocess.py: [Open preprocess.py](#)
- model.pt: [Open model.pt](#)
- Hugging Face Dataset: [Open Dataset](#)

2 Introduction

The goal of this project was to predict the GPS location of an image using deep learning. Our team developed an image-to-location regression model that takes a photograph as input and predicts latitude and longitude coordinates. Unlike image classification tasks with discrete labels, this project required continuous spatial prediction, making it a more challenging computer vision and regression problem.

Our team designed a ResNet50-based regression model trained on a custom dataset of geotagged images collected from mobile devices. To improve performance, we experimented with coordinate normalization, local coordinate transformations, augmentation strategies, loss function design, and transfer learning techniques. Our final model achieved approximately 30 meters average Haversine distance on our internal evaluation dataset and approximately 76 meters in the hidden leaderboard dataset, outperforming the provided baseline model.

One major finding from this project was the difficulty of achieving strong generalization across datasets. While the model performed strongly on internal split validation data, leaderboard performance demonstrated that geographic image prediction is highly sensitive to distribution shifts such as lighting conditions, framing, environmental variation, and differences in camera hardware. Through iterative experimentation, we found that stronger augmentation reduced overfitting, and significantly improved coordinate normalization, which improved robustness.

In addition to developing a competitive leaderboard submission, this project also focused on understanding the tradeoff between strong internal validation performance and real-world generalization. Throughout development, we iteratively refined preprocessing pipelines, augmentation strength, coordinate normalization strategies, and optimization procedures to study how each design choice affected both validation and hidden-test performance. This project, therefore, combined both applied machine learning engineering and experimental analysis.

3 Core Components

3.1 Data Collection and Dataset

Our dataset consisted of geotagged images collected from personal mobile devices. The specified test region was divided into a northern, middle, and southern region, with each team member tasked with collecting at least 8 images from a minimum of 42 spots in each region. Between all 3 members, this would total to 1,008 images for the dataset. This method was used to ensure that our dataset had near equal coverage of the entire test region without data-biasing areas that the team members frequented more often. We extracted GPS

metadata from EXIF information and constructed a dataset containing image paths, latitude coordinates, and longitude coordinates. The dataset ultimately contained approximately 1,115 JPEG images. To improve reproducibility and accessibility, the final dataset was uploaded as a Hugging Face Dataset repository. The dataset includes image files and associated metadata CSVs containing image paths and GPS coordinates. This allowed training and evaluation pipelines to directly stream the dataset during experimentation.

Images were standardized into RGB format and resized before training. We used a 70/15/15 split for training, validation, and testing. The split was performed randomly but reproducibly using a fixed random seed.

Instead of directly regressing latitude and longitude values, we transformed coordinates into a local Cartesian coordinate system centered around the mean training latitude and longitude. Specifically, each GPS coordinate was converted into local (x,y) meter offsets relative to the training origin. This reduced scale imbalance between predictions and stabilized optimization by transforming the relatively small geographic coordinate range into a more numerically stable local coordinate space.

We additionally normalized the local coordinate targets using the training set mean and standard deviation:

$$x' = \frac{x - \mu_x}{\sigma_x}, \quad y' = \frac{y - \mu_y}{\sigma_y}$$

This normalization improved convergence speed and numerical stability during training. Coordinate normalization also improved gradient scaling during optimization. Without normalization, the model frequently produced unstable updates due to the relatively small geographic coordinate range present in the dataset. Converting predictions into normalized local coordinate space simplified the regression objective and reduced sensitivity to learning rate selection.

3.2 Model Design

Our final model used a ResNet50 backbone initialized with ImageNet pretrained weights. We replaced the classification head with a regression head consisting of fully connected layers, batch normalization, ReLU activations, and dropout regularization. ResNet50 was selected due to its strong balance between representational capacity, training stability, and computational efficiency relative to larger backbone architectures.

The final regression head architecture was:

$$2048 \rightarrow 1024 \rightarrow 256 \rightarrow 2$$

where the final two outputs corresponded to normalized local (x,y) coordinate predictions.

Initially, we froze most backbone layers and trained only the final layers. After several epochs, we unfroze the entire network and fine-tuned the model with a smaller learning rate. This transfer learning strategy improved convergence and reduced instability during early training. We additionally experimented with different backbone freezing schedules and learning rate scaling strategies during fine-tuning. Earlier experiments that fully unfroze the network from the beginning frequently converged to poorer local minima and produced substantially higher validation error.

We experimented extensively with augmentation techniques to improve generalization. Our final augmentation pipeline included:

- Random resized cropping
- Horizontal flipping
- Color jitter
- Random perspective distortion

- Random erasing

We found that augmentation strength had a major effect on leaderboard performance. Models trained with weaker augmentation often achieved very low validation error on internal dataset splits but generalized poorly to the hidden evaluation dataset, suggesting substantial overfitting to environmental and viewpoint characteristics present in the collected training images. Overall, augmentation was one of the most important regularization tools in our final pipeline.

Our final loss function combined the normalized regression loss with the geographic distance loss:

$$L = L_{\text{Huber}} + \lambda L_{\text{Haversine}}$$

where the Haversine component directly penalized geographic prediction error in meters by a weight factor (λ) set to 0.05. This helped align optimization with the evaluation metric used on the leaderboard.

3.3 Evaluation and Model Performance

We evaluated our model using the mean Haversine distance measured in meters. Haversine distance computes the geographic distance between predicted and ground truth GPS coordinates on the Earth’s surface.

During development, we evaluated model performance on our internal validation split after every epoch. We tracked both normalized regression loss and mean geographic distance. Early stopping was used to reduce overfitting. Model checkpoints were selected based on validation Haversine distance rather than training loss. This was important because lower regression loss did not always correspond to improved geographic accuracy, particularly during later training epochs where overfitting became more pronounced.

Our internal experiments demonstrated that stronger augmentation and coordinate normalization significantly improved generalization. Early versions of the model achieved approximately 80–100 meter validation error, while the final model achieved approximately 30 meters on our internal test split.

However, leaderboard performance on the hidden dataset was consistently worse, typically around 70-80 meters. We believe one cause for this discrepancy resulted from lacking a dataset that provided finer coverage of the test region and more images at each location. There could have also been a distribution shift between our collected dataset and the hidden evaluation set, such as environmental conditions and viewpoints.

Despite this gap, our final submission consistently outperformed the provided baseline model and demonstrated strong performance relative to the complexity of the task. One limitation of our approach is that the dataset remained relatively small for a geographic localization task. The model therefore struggled to generalize to unseen environmental conditions and viewpoints present in the hidden leaderboard dataset. Future work could incorporate larger-scale datasets, contrastive pretraining, or transformer-based architectures to improve robustness.

Model Variant	Validation Distance (m)	Leaderboard Distance (m)
Baseline Model	~100	~120
Initial ResNet50	~80	~95
Augmented ResNet50	~45	~82
Final Model	~30	~76

Table 1: Performance comparison across model iterations.

These experiments demonstrated that architectural improvements alone were insufficient to achieve strong hidden-set performance. Generalization-focused improvements, such as augmentation tuning, coordinate normalization, and transfer learning strategy contributed more substantially to leaderboard gains than simply increasing model complexity.

4 Exploratory Components

4.1 Technique Exploration

One exploratory direction focused on improving coordinate prediction through local coordinate transformations instead of directly predicting latitude and longitude. Direct regression on GPS coordinates often produced unstable optimization due to differences in geographic scaling.

To address this, we converted GPS coordinates into local Cartesian coordinates relative to the dataset origin. This reduced the numerical scale of predictions and simplified the learning objective.

We additionally explored blended loss functions that combined normalized coordinate regression with Haversine distance loss. Ablation experiments showed that including geographic distance directly in the loss improved final evaluation performance.

Beyond our core ResNet-50 pipeline, we pursued two modeling extensions aimed at improving prediction accuracy: k-fold cross-validation and an ensemble combining ResNet-50 with EfficientNet-B0. Both directions were implemented and evaluated, but ultimately neither was adopted in our final submission due to unfavorable performance-to-cost tradeoffs.

K-Fold Cross-Validation. With a training dataset of approximately 1,115 images, we were concerned that a single fixed train/validation split might produce an unreliable estimate of generalization performance. We implemented 5-fold cross-validation to make fuller use of available data, training five independent ResNet-50 models on different 80% subsets of the training corpus and selecting the best checkpoint across all folds. While this approach produced a more stable estimate of validation Haversine distance and reduced variance in our performance measurements, the improvement over some of our initial single-fold implementations was marginal in practice. The performance evaluated by the leaderboard also remained low, barely improving over the baseline by a couple meters. More significantly, training five full models multiplied training time by approximately $5\times$, which severely limited our ability to iterate on other design decisions such as augmentation strength, learning rate schedules, and head architecture. Given that the compute cost was not justified by the leaderboard error, we reverted to a single dataset split for our final submission and instead invested that compute budget in hyperparameter tuning.

EfficientNet-B0 Ensemble. Motivated by the observation that different backbone architectures learn complementary visual features, we implemented an ensemble combining ResNet-50 with an EfficientNet-B0 backbone. Both models were trained independently on the same dataset, and their predictions were combined by averaging in normalized local coordinate space before converting back to latitude and longitude. The theoretical motivation was that averaging two independent predictors in the same coordinate space should reduce prediction variance without introducing systematic bias.

In practice, however, EfficientNet-B0 performed substantially worse than ResNet-50 on our dataset. We attribute this to two factors. First, EfficientNet-B0 is a significantly smaller model (approximately 5.3 million parameters compared to ResNet-50’s 25 million), and its representational capacity may have been insufficient for the fine-grained geographic discrimination required by this task. Second, and more importantly, EfficientNet-B0 failed to produce well-calibrated latitude and longitude predictions, with its outputs showing high geographic error relative to ResNet-50. Because the ensemble averaged the two models’ coordinate predictions with equal weight, EfficientNet-B0’s poor predictions actively degraded the ensemble output below ResNet-50’s standalone performance rather than complementing it. Attempts to reweight the ensemble in favor of ResNet-50 recovered some performance but never exceeded the single-model baseline, suggesting our ensemble was net-negative for this task and dataset size. We concluded that ensemble methods are most beneficial when constituent models are independently strong and make different errors — a condition that was not met here, given the performance gap between the two backbones.

4.2 Code Exploration

To maximize iteration speed under our compute constraints, we implemented three pipeline optimizations that together reduced per-epoch training time substantially, allowing more experimental iterations within

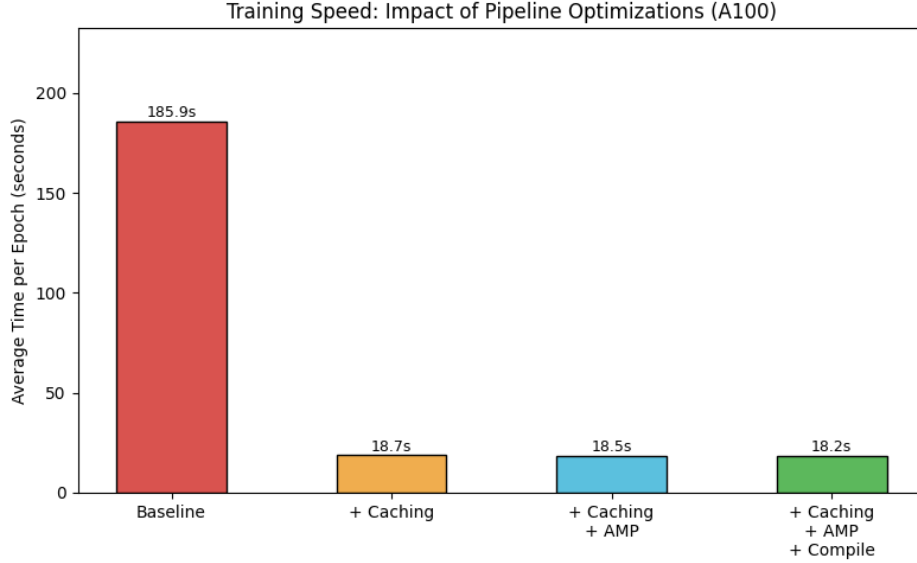


Figure 1: Training speed improvement through added optimizations.

our A100 GPU allocation.

The first optimization was in-memory image caching in `GPSTImageDataset`. Rather than decoding images from the Hugging Face dataset on every training step, all images are resized to 448×448 pixels and stored as `uint8` tensors in RAM at dataset construction time, with augmentations applied on-the-fly to the cached tensors during training. This eliminated repeated JPEG decoding and dataset deserialization, which were the dominant I/O bottlenecks. Caching at 448 pixels rather than the final training resolution of 384 ensures `RandomResizedCrop` always has sufficient pixel information regardless of crop scale.

The second optimization was mixed precision training using `torch.amp.autocast` with `bfloat16` dtype. The `bfloat16` format was chosen over `float16` for its wider dynamic range, which improved gradient stability during the early frozen-backbone phase. On the A100, `bfloat16` tensor core acceleration reduced forward and backward pass time relative to `float32`. This was further complemented by channels-last memory layout and TF32 matrix multiplication, both of which enable more efficient use of the A100’s hardware.

Finally, `torch.compile` was applied before training to trigger operator fusion across the forward pass, reducing Python interpreter overhead in the training loop. One practical consequence was that compiled checkpoints prefix state dict keys with `_orig_mod.`, requiring an explicit key-stripping step in our export cell to ensure compatibility with the leaderboard backend.